

Gaze Prompts: Temporally Dense Human Attention for Vision–Language–Action Fine-Tuning

Yihan Zhou^{1,2,3*}, Rui Yan^{1*}, Mingcong Li⁴, Zheyuan Huang⁴,
Xu Yang¹, Xueyang Guo^{1,2,3}, Yilin Mo^{1,2,3*}

¹Department of Automation, Tsinghua University

²Beijing Key Laboratory of Embodied Intelligence Systems

³Institute for Embodied Intelligence and Robotics, Tsinghua University

⁴LingYu Robotics

*Equal Contribution

*Corresponding Author

{zhouyh23, yan-r22}@mails.tsinghua.edu.cn, ylmo@tsinghua.edu.cn

Abstract: Vision–Language–Action (VLA) fine-tuning pairs images with actions at every step, yet typically provides only a task-level language instruction, leaving moment-to-moment visual relevance implicit. We introduce *eye-tracker-supervised gaze prompting*, which uses gaze recorded during VR teleoperation to provide frame-level visual guidance for VLA fine-tuning. During training, recorded gaze locations are rendered as crosshairs on the robot’s head-camera images. At deployment, a lightweight predictor estimates gaze locations from recent images and the instruction, supplying the same type of visual prompt without an eye tracker or changes to the policy architecture. Instantiated with π_0 , gaze prompting increases mean success from 26.3% to 56.0% across six real-world bimanual manipulation tasks, with gains also observed when a single policy is trained on all six tasks. We release GAZEMANI, a dataset of 1,200 teleoperated trajectories with synchronized gaze.

Keywords: Vision–Language–Action Models; Visual Prompting; Human Gaze; Robot Manipulation

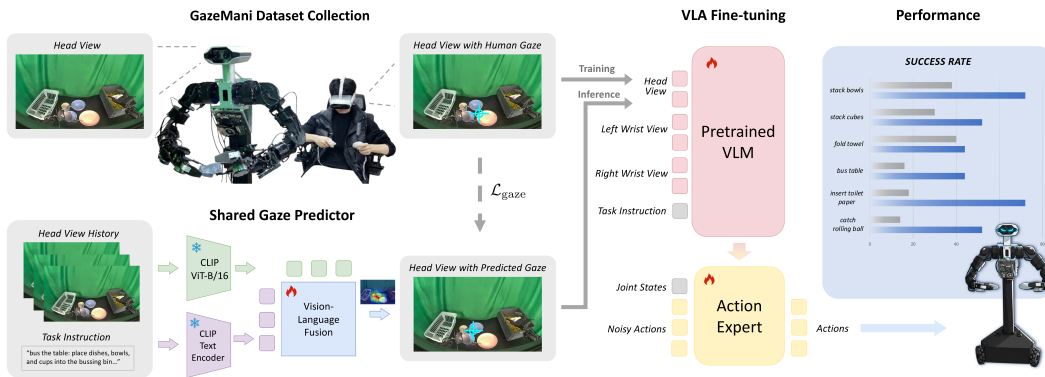


Figure 1: **Eye-tracker-supervised gaze prompting.** During VR teleoperation (upper left), an HMD eye tracker records the operator’s gaze, which supervises a lightweight gaze predictor (lower left) and is rendered as a crosshair on the head-camera image for VLA fine-tuning (center). At deployment, the predictor estimates gaze from recent head-camera frames and the language instruction, providing the same visual prompt to the fine-tuned VLA without an eye tracker or architectural modifications. The right panel summarizes task performance.

1 Introduction

Vision–Language–Action (VLA) models couple pretrained vision–language backbones with large-scale robot pretraining [1, 2, 3, 4], and are then adapted to a new task or platform by fine-tuning on a limited number of demonstrations. The supervision available during fine-tuning, however, arrives at two very different granularities: each demonstration pairs images with an action at every frame, while its language instruction specifies the goal only once. Successful execution requires frame-by-frame choices about which object instance to attend, which task phase is active, and where contact will occur next, yet neither signal explicitly annotates these frame-level decisions. What is missing is a temporally dense trace of what the demonstrator considered relevant at each moment.

The consequences of this gap are increasingly well documented. Mechanistic analyses show that language does not uniformly drive VLA action generation: actions depend predominantly on visual and spatial features, with instruction changes affecting behavior mainly when the scene alone underdetermines the correct action [5]. Fine-tuning can reinforce this imbalance, since robot data offers dense visual–action supervision but little linguistic variation, encouraging shortcut learning and language forgetting [6]. Existing remedies inject spatial structure through visual prompts, auxiliary losses, or attention modifications—but their cues are engineered or model-derived rather than measured from the demonstrator’s own attention.

The missing trace is naturally available in VR teleoperation: the operator’s headset records their gaze alongside each demonstration through its integrated eye tracker. Prior studies of natural behavior have shown that gaze is closely coupled to action, with fixations selecting task-relevant objects, shifting across sub-task boundaries, and anticipating contact before the hand arrives [7, 8]. Yet this signal is available only during data collection, since the deployed robot carries no eye tracker. Using gaze at deployment therefore requires predicting it from the robot’s own observations.

We introduce *eye-tracker-supervised gaze prompting* for VLA fine-tuning (Figure 1). During fine-tuning, the recorded gaze location is rendered as a crosshair on the VLA’s RGB input. A lightweight gaze predictor (~ 2.9 M trainable parameters) is trained independently to map RGB observations and language to the gaze location, providing the gaze signal without an eye tracker at deployment. This converts gaze recorded during demonstration collection into a deployable visual cue for the VLA’s existing image pathway.

Overall, our contributions are summarized as follows:

1. We use human gaze recorded during teleoperation as temporally dense supervision for VLA fine-tuning, and learn a lightweight predictor that provides gaze prompts to an unchanged policy without an eye tracker or architectural modification at deployment.
2. Experiments on six real-world bimanual manipulation tasks validate the effectiveness of gaze prompting for VLA fine-tuning, with mean success increasing from 26.3% to 56.0% and comparable gains under unified multi-task training.
3. Controlled route and cue-source analyses characterize the contribution of inference-time gaze cues and show that human gaze can indicate upcoming task-relevant locations.

We release the GAZEMANI dataset, comprising 1,200 bimanual trajectories with synchronized gaze recordings across six tasks, along with evaluation code at <https://gazemani.github.io/>.

2 Related Work

Language conditioning in VLA fine-tuning. VLA policies inherit broad semantic knowledge from pretrained vision–language models, yet recent studies suggest that language does not always play a central role in action generation. Mechanistic analyses show that actions can rely heavily on visual and spatial features, while language becomes more influential when the scene alone leaves multiple plausible actions [5]. Fine-tuning can further amplify this imbalance: robot datasets provide dense visual–action supervision but comparatively little linguistic variation, encouraging

shortcut learning, information collapse, and language forgetting [6, 9]. Other work addresses this mismatch through guidance, decomposition, or alignment objectives [10, 11, 12]. Our work takes a complementary approach by augmenting VLA fine-tuning with a temporally dense signal of the demonstrator’s visual attention, rather than modifying the language objective or adding a reasoning module.

Spatial and attentional cues for VLA policies. Existing methods inject task-relevant visual structure into VLA policies through several routes. Auxiliary-loss methods regularize internal attention toward gaze, saliency, or grounded regions [13, 14]; architectural methods modify how attention is computed [15, 16]; and inference-time guidance methods use modified visual inputs to stabilize object-centric action generation [17]. A closely related line renders spatial cues directly onto the RGB observation. TraceVLA draws past motion traces [18]; AimBot renders scope reticles encoding end-effector pose [19]; HAMSTER and RT-Affordance provide VLM- or affordance-derived paths and waypoints [20, 21]; and VP-VLA renders planner-generated crosshairs and boxes [22]. Across these approaches, the attentional or spatial guidance is typically engineered or model-derived, drawing on robot geometry, scene heuristics, external planners, model internals, or synthetic gaze rather than the human demonstrator’s gaze.

Human gaze as an attentional signal. Studies of natural behavior show that gaze is tightly coupled to action: fixations select task-relevant objects, shift across sub-task boundaries, and anticipate upcoming contact [8, 23, 24]. This has motivated imitation-learning methods that use gaze through feature masks, gaze-centered crops, auxiliary losses, or additional input channels [25, 26, 27, 28], as well as more recent robot and VLA methods for foveated tokenization, attention regularization, point-cloud cropping, learned eye policies, and subtask decomposition [29, 13, 30, 31]. Our work connects prior work on human gaze with pixel-space visual prompting: gaze recorded during teleoperation provides supervision for a lightweight predictor, whose predicted gaze location is delivered as a per-frame RGB prompt through the existing visual pathway of an unchanged VLA policy.

3 Method

Our method turns gaze recorded during teleoperated demonstrations into a deployable pixel-space prompt for a VLA policy. During policy fine-tuning, the recorded gaze location is rendered as a crosshair on the head-camera image. Separately, a lightweight predictor learns to estimate gaze from recent head-camera frames and the language instruction; at inference, its predicted gaze location is rendered in the same way, providing a per-frame gaze cue without an eye tracker. The policy architecture is unchanged, and the gaze cue reaches action generation only through the same visual pathway that processes the rest of the image.

The pipeline has three components. A gaze predictor ($\sim 2.9\text{M}$ trainable parameters) maps a short window of N head-camera frames and the language instruction to a gaze heatmap over the current view (§3.1). A renderer converts a gaze location into a crosshair on the head-camera image, while wrist-camera views remain unchanged (§3.2). A VLA policy, instantiated as π_0 [3] in our experiments, is fine-tuned on crosshair-rendered observations, consumes the resulting multi-view input, and emits actions. The predictor and policy are trained separately (§3.3).

3.1 Gaze predictor

The predictor maps N consecutive head-camera frames $\mathbf{I}_{t-N+1:t}$ and a language instruction ℓ to a 224×224 gaze heatmap over the current view (Figure 2); we use $N = 3$ throughout.

Architecture. A frozen CLIP ViT-B/16 [32] vision encoder processes each frame independently into 196 visual tokens (one per 16×16 patch); a frozen CLIP text encoder embeds the instruction into T text tokens. Both token sequences are linearly projected to a shared 256-dimensional fusion space. We keep the two CLIP encoders frozen and train everything else—the projections, the prompt-

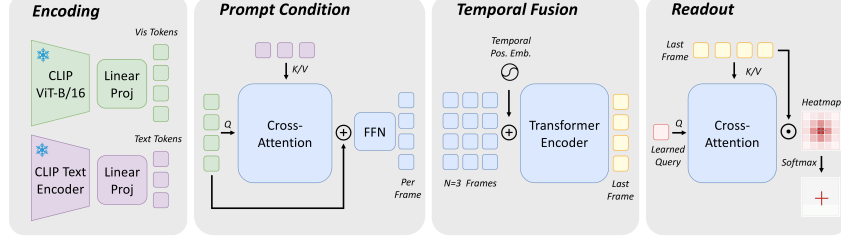


Figure 2: Gaze predictor architecture. Frozen CLIP encoders encode each head-camera frame and the instruction, and learned linear projections map their tokens to a shared fusion space (**Encoding**); each frame’s visual tokens cross-attend to the text tokens (**Prompt conditioning**); a transformer fuses the $N=3$ prompt-conditioned frames and retains the last frame’s tokens (**Temporal fusion**); these are reshaped to a 14×14 grid and upsampled to a 224×224 gaze heatmap (**Readout**). The architecture details are given in §3.1.

conditioning and temporal-fusion blocks, and the readout decoder. After encoding, the architecture has three trainable stages:

1. *Per-frame prompt conditioning.* Each frame’s visual tokens cross-attend to the text tokens (vision queries text), followed by a feed-forward network. This injects instruction semantics into every frame’s spatial representation before any temporal mixing.
2. *Temporal fusion.* The N prompt-conditioned token grids ($N \times 196$ tokens) are augmented with learnable temporal positional embeddings and processed by a 2-layer transformer encoder. We extract the last frame’s contextualized tokens as the output; these now carry information from the preceding $N-1$ frames.
3. *Readout.* The last frame’s 196 contextualized tokens are reshaped to a 14×14 feature map and decoded by a learned transposed-convolution upsampler into 224×224 logits, one score per output cell. Decoding to 224×224 provides a denser spatial readout than predicting gaze coordinates directly on the coarse 14×14 grid.

Supervision. The ground-truth gaze location is the operator’s eye-tracker gaze registered to the head-camera view and temporally filtered during dataset preprocessing. Each frame’s gaze location is rasterized into a normalized Gaussian-smoothed 224×224 heatmap G^* (smoothing σ ablated in Appendix F). The predictor is supervised with per-frame cross-entropy:

$$\mathcal{L}_{\text{gaze}} = - \sum_{i,j} G_{ij}^* \log \hat{G}_{ij}, \quad \hat{G} = \text{softmax}(f_{\theta}(\mathbf{I}_{t-N+1:t}, \ell)), \quad (1)$$

where f_{θ} denotes the full predictor producing unnormalized 224×224 logits. The predictor is trained jointly on demonstrations from all six tasks, exposing it to a broader range of task and visual contexts than any single task. Because per-task episode lengths differ substantially, we use a task-balanced weighted sampler so that each of the six tasks receives equal sampling probability and longer tasks do not dominate training.

3.2 Visual-prompt rendering

The renderer draws a crosshair at a given gaze location on the head-camera image, with fixed geometry and color across all tasks and instructions (details in Appendix G). Only the head view is modified; wrist views pass through unchanged. During policy fine-tuning, the crosshair is rendered at the ground-truth gaze. At inference, the gaze location is obtained from the predictor’s 224×224 heatmap by applying a soft-argmax over the 3×3 neighborhood of its peak, followed by causal exponential moving-average (EMA) smoothing of the resulting coordinates.



Figure 3: **The GAZEMANI task suite.** Six real-world bimanual manipulation tasks used for policy evaluation; the cyan crosshair is the rendered gaze prompt.

3.3 Training and inference

The predictor and policy are trained independently, with no gradients flowing between them. The predictor is supervised by ground-truth gaze locations via Eq. (1). The policy is fine-tuned with the crosshair rendered at the ground-truth gaze location, while the action target remains the original demonstrated action. At inference, the frozen predictor estimates gaze from a rolling buffer of the N most recent head-camera frames, and the VLA policy acts on the resulting crosshair-rendered observation. The predictor adds one forward pass per policy step (latency in Appendix J).

4 Experimental Results

4.1 Setup

Task suite. We evaluate on six real-world bimanual manipulation tasks (Figure 3; full descriptions in Appendix C). The suite spans instruction-conditioned instance selection (*Stack Bowls*, red or yellow), phase-dependent sequencing (*Stack Cubes in Order*), long-horizon tidying (*Bus Table*), deformable-state tracking (*Fold Towel*), precise contact localization (*Insert Toilet Paper*), and dynamic interception (*Catch Rolling Ball*).

Robot platform. We use TeleAvatar, a bimanual mobile manipulator from LingYu Robotics, equipped with two 7-DoF arms, one-DoF grippers, a head-mounted stereo fisheye camera, and two wrist cameras. The platform is used for both teleoperated data collection and policy deployment (Figure 4); additional hardware details are provided in Appendix A.



Figure 4: Experimental platform, used for both teleoperated data collection and policy deployment. *Left:* the bimanual mobile manipulator—two 7-DoF arms with one-DoF grippers on a torso column above a wheeled base, with a head-mounted stereo fisheye camera. *Right:* the Pico 4 Enterprise headset and its two controllers, used only during data collection for VR teleoperation and eye tracking—no headset or eye tracker is needed at deployment.

Data collection (GAZEMANI). Demonstrations are collected via VR teleoperation using a Pico 4 Enterprise headset with an integrated eye tracker. The recorded 3D gaze rays are registered offline to 2D head-camera image coordinates (Appendix B), and the resulting gaze trace is temporally smoothed with a Kalman filter. Each frame records three RGB views, the robot’s proprioceptive joint

Table 1: Main policy results across six manipulation tasks. Cells report success rate over 50 rollouts (best per task in bold). Parenthesized values are the change over vanilla in percentage points (**green**: gain, **red**: loss); the last column is the headroom-normalized gain $\Delta_{\text{norm}} = \Delta\text{SR}/(1 - \text{SR}_{\text{vanilla}})$ of gaze-prompt over vanilla.

Task	Description	Vanilla	Aux-loss	Gaze-prompt	Δ_{norm}
T1	Stack Bowls	38%	60% (+22)	72% (+34)	55%
T2	Stack Cubes in Order	30%	44% (+14)	52% (+22)	31%
T3	Bus Table	16%	8% (−8)	44% (+28)	33%
T4	Fold Towel	42%	20% (−22)	44% (+2)	3%
T5	Insert Toilet Paper	18%	44% (+26)	72% (+54)	66%
T6	Catch Rolling Ball	14%	24% (+10)	52% (+38)	44%
Average		26.3%	33.3% (+7.0)	56.0% (+29.7)	—

state, the demonstrated action, and the operator’s gaze location. Each task provides 200 demonstrations (1,200 total); per-task counts and episode-length statistics are reported in Appendix D.

Methods. We compare three methods that share the same π_0 backbone and demonstration set (training and hyperparameter details in Appendix E):

- **Vanilla** π_0 : standard fine-tuning on unmodified RGB observations; no gaze signal.
- **Aux-loss** π_0 : the Gaze-Regularized VLA mechanism [13], which adds an auxiliary KL loss regularizing the policy’s language→vision attention toward the gaze location while leaving the architecture and action objective unchanged. We replace the original synthetic gaze with our recorded human gaze and keep the original KL coefficient. The gaze signal is available only during training.
- **Gaze-prompt** π_0 (**ours**): π_0 fine-tuned on head-view images with a crosshair rendered at the recorded gaze location, and evaluated with a crosshair rendered at the predicted gaze location. The policy architecture is unchanged.

We fine-tune a separate policy for each task in the main experiments and additionally evaluate a single policy trained jointly across all six tasks in §4. Object placements are randomized across rollouts, with fixed seeds for stochastic policy components; rollout counts are reported within each experiment.

4.2 Policy performance

Table 1 reports the main results with a separately fine-tuned policy for each task. Gaze-prompt π_0 improves over vanilla on all six tasks, increasing mean success from 26.3% to 56.0% (+29.7 points), and outperforms aux-loss on every task. The largest gains over vanilla occur on T1, T5, and T6, where successful execution requires selecting or precisely localizing small, visually ambiguous, or moving targets; the gain is smallest on Fold Towel (T4), where vanilla already reaches 42%. Two-sided Fisher exact tests find the improvement over vanilla significant on five of the six tasks, with T4 as the exception. To account for differences in baseline performance, the last column additionally reports the headroom-normalized gain $\Delta_{\text{norm}} = \Delta\text{SR}/(1 - \text{SR}_{\text{vanilla}})$. Uncut single-take recordings of all rollouts reported in Table 1 are available on the project website.

The two longest tasks, T3 and T4, further show how performance evolves within a rollout (Figure 5). On Fold Towel, gaze-prompt remains close to vanilla across the sequence, consistent with their similar final success rates. On Bus Table, the advantage of gaze-prompt is more pronounced on the later, harder items, sorting 4.8 of 7 items on average compared with 3.4 for both baselines.

The gain also persists when all six tasks are learned by a single policy (Figure 6). Gaze-prompt achieves the highest observed success rate on all six tasks and reaches 49.2% pooled success over 120 rollouts, compared with 20.8% for vanilla and 29.2% for aux-loss. Under unified training, its improvement over vanilla remains comparable to that under per-task training. These results show that the benefit of gaze prompting is not specific to training an independent policy for each task.

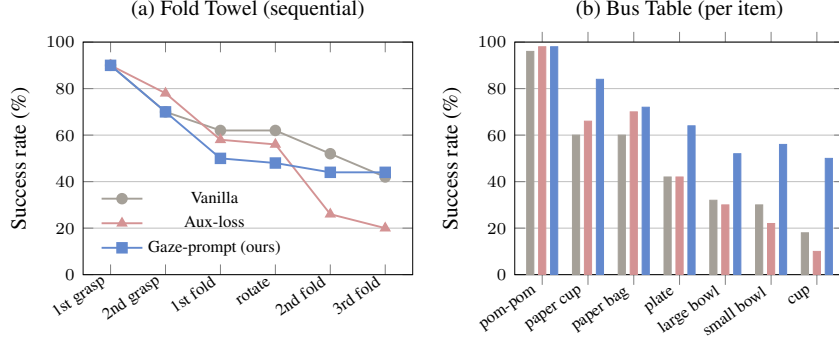


Figure 5: **Stage-wise success on the long-horizon tasks.** (a) Fold Towel, per fold stage; (b) Bus Table, per item.

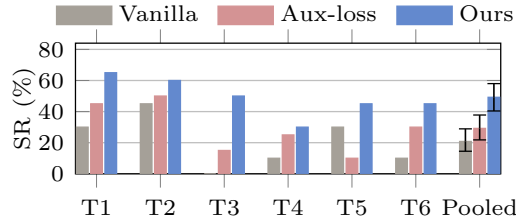


Figure 6: **Unified multi-task policy performance.** Each method is trained as a single policy jointly across all six tasks. Bars show observed per-task success rates over 20 rollouts. Pooled results aggregate 120 rollouts per method and include 95% Wilson confidence intervals.

4.3 Injection route and inference-time cue

The two gaze-supervised variants incorporate human gaze in different ways. *Aux-loss* uses gaze through a training-time attention objective, whereas *gaze-prompt* renders the gaze location directly in the visual input and continues to provide the predicted cue at inference. To examine the contribution of this inference-time cue, we evaluate the same T1 *gaze-prompt* policy with the crosshair removed only at inference. Success decreases from 72% to 58% over 50 rollouts, close to the 60% achieved by *aux-loss*. The observed drop is consistent with a contribution from the fresh per-frame cue available at inference.

An instruction-swap analysis on *Stack Bowls* (T1) further shows how this cue affects the policy’s response to language. The visual scene is kept unchanged while the instruction switches between stacking the red and yellow bowls. Under this controlled swap, vanilla π_0 distributes attention broadly across the tabletop, whereas the gaze-supervised variants show stronger instruction-dependent shifts in attention (Figure 7; extraction details in Appendix H). *Gaze-prompt* exhibits the largest instruction-conditioned shift, with a Jensen–Shannon (JS) divergence of 0.117 between the two attention maps, compared with 0.068 for *aux-loss* and 0.047 for vanilla. Corresponding task success is reported in Table 2.

4.4 Cue source and anticipation

To examine how the source of the visual cue affects policy performance, we compare five variants on T6 (*Catch Rolling Ball*) under the same input-pixel rendering route. Each prompted variant independently fine-tunes π_0 on its corresponding cue. T6 provides a clean setting for this comparison because it contains a single moving target, avoiding an additional object-selection rule when constructing engineered cues. We compare predicted human gaze, a Kalman-extrapolated future-position cue, current-position detection, a random crosshair, and no cue. For the two position-based cues, we use the same CPU color-blob detector on the policy’s 224×224 input. It runs in 4.06 ms,

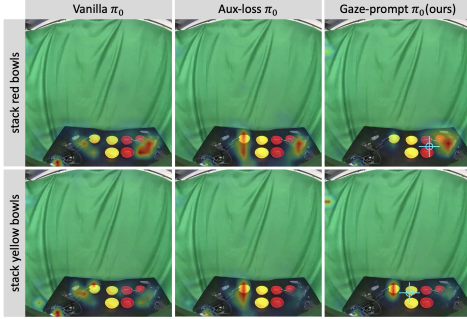


Figure 7: T1 instruction-swap attention.

Table 2: T1 instruction-swap results.

Method	SR_A	SR_B	JS
Vanilla π_0	40%	36%	0.047
Aux-loss π_0	60%	60%	0.068
Gaze-prompt π_0	84%	60%	0.117

A: stack red bowls; B: stack yellow bowls

Table 3: Cue-source ablation on T6. All prompted variants use the same input-pixel rendering route. Brackets report Wilson 95% confidence intervals over 50 rollouts.

Cue source	Success rate
Human-gaze predictor (ours)	52% [38.5, 65.2]
Kalman future-position cue	38% [25.9, 51.8]
Current-position detection	12% [5.6, 23.8]
Random crosshair	10% [4.3, 21.4]
No cue	14% [7.0, 26.2]

close to the gaze predictor’s 3.8 ms, and places the cue on the ball in 94.5% [90.4, 96.9] of 200 manually checked pre-grasp frames. The Kalman cue extrapolates the detected trajectory 541 ms ahead, matching the mean interval from arm-movement onset to the ball’s arrival at the movement-onset gaze location.

Neither current-position detection nor the random crosshair differs significantly from the no-cue baseline. By contrast, extrapolating the ball’s future position raises success from 12% to 38% relative to current-position detection ($p=0.005$), showing that predictive position information is useful for interception. Predicted human gaze achieves the highest observed success rate at 52%. Figure 8 provides a qualitative view of the corresponding attention patterns.

The timing analysis further distinguishes human gaze from the tested kinematic predictor. The ball reaches the movement-onset gaze location approximately 541 ms after arm movement begins, defining the prediction horizon used by the Kalman cue. Human gaze settles near this location an average of 949 ms before movement onset. When predicting the same ball-arrival event from gaze-dwell onset and movement onset, the Kalman predictor yields mean errors of 420 and 118 px, respectively.

A similar anticipatory pattern appears in the long-horizon Bus Table task (T3). Over more than 1,000 aligned grasp and release events, gaze settles near the next object a median of 1667 ms before gripper closure and near the future placement site a median of 933 ms before gripper opening; at that time, the manipulated object has not yet reached the placement location. These measurements complement the T6 ablation: human gaze is not merely a marker of the currently visible target, but can also encode upcoming task-relevant locations before they become the current manipulation target.

5 Limitations

While effective across the six evaluated manipulation tasks, gaze prompting has several limitations:

- **Dependence on gaze prediction quality.** The inference-time prompt is produced by a learned gaze predictor, so inaccurate predictions can place the crosshair at irrelevant or misleading locations. Although the predictor attains about 10-pixel held-out localization error on the 224×224 policy input, we do not yet characterize systematic or high-confidence prediction errors, nor do we use prediction confidence to suppress unreliable cues.

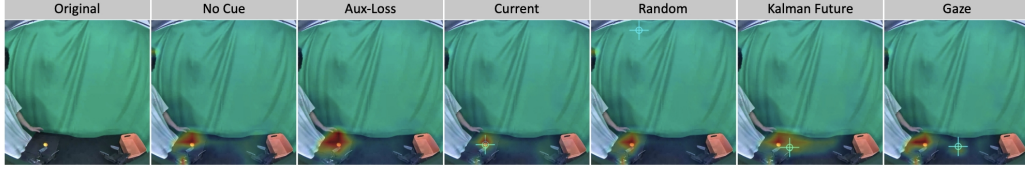


Figure 8: **Representative attention maps on T6.** Columns show the original frame and attention maps for No Cue, Aux-Loss, Current, Random, Kalman Future, and Gaze. Aux-Loss uses human gaze only during training and is shown for reference; the remaining prompted variants use rendered inference-time cues. “Kalman Future” extrapolates the detected ball trajectory 541 ms ahead. Cyan crosshairs denote rendered cues.

- **Generalization beyond the evaluated setting.** Our experiments use a single robot embodiment, π_0 backbone, and data-collection setup. Only T1 varies the language instruction within a task, while T2–T6 use a fixed instruction. The current results therefore do not establish generalization to unseen instructions, novel tasks, substantially different visual domains, other robot embodiments, or other VLA backbones.
- **Scope of predictive baselines.** The latency-matched current- and future-position baselines are evaluated on T6, where the moving target admits a well-defined kinematic extrapolation. T3 provides complementary timing evidence that gaze can anticipate upcoming object and placement targets, but analogous predictive baselines are less straightforward for long-horizon or multi-object manipulation. A broader characterization of the future task information encoded by gaze remains open.

6 Conclusion

We introduced *eye-tracker-supervised gaze prompting*, which turns human gaze recorded during VR teleoperation into a visual prompt for VLA fine-tuning and deployment. A lightweight gaze predictor supplies the cue at inference, allowing an unchanged π_0 policy to use the same input-pixel prompt without an eye tracker or architectural modification. Across six real-world bimanual manipulation tasks, gaze prompting increases mean success from 26.3% to 56.0% under per-task training and retains comparable gains when all six tasks are learned by a single policy.

Controlled analyses further characterize the role and content of the cue. Removing the crosshair only at inference reduces performance, while instruction-swap experiments show stronger instruction-dependent attention under gaze supervision. On the dynamic interception task, predicting the ball’s future position is substantially more effective than marking its current position, and predicted human gaze achieves the highest observed success. Timing analyses on both dynamic and long-horizon tasks further show that gaze can indicate upcoming task-relevant locations before they become the current manipulation target. These results support human gaze as a practical source of temporally dense supervision that can be collected alongside demonstrations and injected through the existing visual pathway of a VLA policy.

Acknowledgments

We thank Lingyu Robotics for providing the data, robotic platforms, and technical support. This work was supported by the National Natural Science Foundation of China under Grants 62461160313, 62273196, and 62192752, and by the BNRist project under Grant BNR2024TD03003.

References

- [1] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.
- [2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [3] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [4] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [5] B. Grant, X. Zhao, and P. Wang. Not all features are created equal: A mechanistic study of vision-language-action models. *arXiv preprint arXiv:2603.19233*, 2026.
- [6] K. Xu, Z. Zhu, A. Chen, S. Zhao, Q. Huang, Y. Yang, H. Lu, R. Xiong, M. Tomizuka, and Y. Wang. Seeing to act, prompting to specify: A bayesian factorization of vision language action policy. *arXiv preprint arXiv:2512.11218*, 2025.
- [7] R. S. Johansson, G. Westling, A. Bäckström, and J. R. Flanagan. Eye–hand coordination in object manipulation. *Journal of neuroscience*, 21(17):6917–6932, 2001.
- [8] M. F. Land and M. Hayhoe. In what ways do eye movements contribute to everyday activities? *Vision research*, 41(25-26):3559–3565, 2001.
- [9] S. Lian, B. Yu, X. Lin, L. T. Yang, Z. Shen, C. Wu, Y. Miao, C. Huang, and K. Chen. Langforce: Bayesian decomposition of vision language action models via latent action queries. In *Proceedings of the 43rd International Conference on Machine Learning*, Proceedings of Machine Learning Research. PMLR, 2026.
- [10] Z. Zhan, Y. Chen, J. Zhou, Q. Lv, H. Liu, K. Wang, L. Lin, and G. Wang. Stable language guidance for vision-language-action models. *arXiv preprint arXiv:2601.04052*, 2026.
- [11] T. Wulff, F. Tavella, R. S. Maharjan, M. Adikari, and A. Cangelosi. Grounding hierarchical vision-language-action models through explicit language-action alignment. *arXiv preprint arXiv:2604.05614*, 2026.
- [12] C.-P. Huang, Y.-H. Wu, M.-H. Chen, F. Wang, and F.-E. Yang. Thinkact: Vision-language-action reasoning via reinforced visual latent planning. *Advances in Neural Information Processing Systems*, 38:82782–82802, 2026.
- [13] A. Pani and Y. Yang. Gaze-regularized vision-language-action models for robotic manipulation. *arXiv preprint arXiv:2603.23202*, 2026.

- [14] W. Song, Z. Zhou, H. Zhao, J. Chen, P. Ding, H. Yan, Y. Huang, F. Tang, D. Wang, and H. Li. Reconvla: Reconstructive vision-language-action model as effective robot perceiver. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 18549–18557, 2026.
- [15] Y. Zhang, W. Yuan, Y. Zhang, X. Zhang, and J. Wan. Focusvla: Focused visual utilization for vision-language-action models. *arXiv preprint arXiv:2603.28740*, 2026.
- [16] L. Xiao, J. Li, J. Gao, F. Ye, Y. Jin, J. Qian, J. Zhang, Y. Wu, and X. Yu. Ava-vla: Improving vision-language-action models with active visual attention. *arXiv preprint arXiv:2511.18960*, 2025.
- [17] J. Zhou, Z. Zhan, R. Zhai, Q. Lyu, H. Liu, K. Wang, L. Lin, and G. Wang. Tag: Target-agnostic guidance for stable object-centric inference in vision-language-action models. *arXiv preprint arXiv:2603.24584*, 2026.
- [18] R. Zheng, Y. Liang, S. Huang, J. Gao, H. Daumé III, A. Kolobov, F. Huang, and J. Yang. Tracevla: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. In *International Conference on Learning Representations*, volume 2025, pages 54277–54296, 2025.
- [19] Y. Dai, J. Lee, Y. Zhang, Z. Ma, J. Yang, A. Zadeh, C. Li, N. Fazeli, and J. Chai. Aimbot: A simple auxiliary visual cue to enhance spatial awareness of visuomotor policies. *arXiv preprint arXiv:2508.08113*, 2025.
- [20] Y. Li, Y. Deng, J. Zhang, J. Jang, M. Memmel, C. Garrett, F. Ramos, D. Fox, A. Li, A. Gupta, et al. Hamster: Hierarchical action models for open-world robot manipulation. In *International Conference on Learning Representations*, volume 2025, pages 24040–24068, 2025.
- [21] S. Nasiriany, S. Kirmani, T. Ding, L. Smith, Y. Zhu, D. Driess, D. Sadigh, and T. Xiao. Rt-affordance: Affordances are versatile intermediate representations for robot manipulation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 8249–8257. IEEE, 2025.
- [22] Z. Wang, Y. Chen, Y. Liu, J. Ye, P. Chen, C. Lu, S. Liu, B. Yu, and J. Jia. Vp-vla: Visual prompting as an interface for vision-language-action models. *arXiv preprint arXiv:2603.22003*, 2026.
- [23] M. Hayhoe and D. Ballard. Eye movements in natural behavior. *Trends in cognitive sciences*, 9(4):188–194, 2005.
- [24] M. F. Land and P. McLeod. From eye movements to actions: how batsmen hit the ball. *Nature neuroscience*, 3(12):1340–1345, 2000.
- [25] R. Zhang, Z. Liu, L. Zhang, J. A. Whritner, K. S. Muller, M. M. Hayhoe, and D. H. Ballard. Agil: Learning attention from human for visuomotor tasks. In *Proceedings of the european conference on computer vision (eccv)*, pages 663–679, 2018.
- [26] H. Kim, Y. Ohmura, and Y. Kuniyoshi. Using human gaze to improve robustness against irrelevant objects in robot manipulation tasks. *IEEE Robotics and Automation Letters*, 5(3): 4415–4422, 2020.
- [27] A. Saran, R. Zhang, E. S. Short, and S. Niekum. Efficiently guiding imitation learning agents with human gaze. *arXiv preprint arXiv:2002.12500*, 2020.
- [28] C. Thammineni, H. Manjunatha, and E. T. Esfahani. Selective eye-gaze augmentation to enhance imitation learning in atari games. *Neural Computing and Applications*, 35(32):23401–23410, 2023.

- [29] I. Chuang, J. Zou, A. Lee, D. Gao, and I. Soltani. Look, focus, act: Efficient and robust robot learning via human gaze and foveated vision transformers. *arXiv preprint arXiv:2507.15833*, 2025.
- [30] J. Kerr, K. Hari, E. Weber, C. M. Kim, B. Yi, T. Bonnen, K. Goldberg, and A. Kanazawa. Eye, robot: Learning to look to act with a bc-rl perception-action loop. *arXiv preprint arXiv:2506.10968*, 2025.
- [31] R. Takizawa, Y. Ohmura, and Y. Kuniyoshi. Gaze-guided task decomposition for imitation learning in robotic manipulation. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 7965–7972. IEEE, 2025.
- [32] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR, 2021.
- [33] L. Beyer, A. Steiner, A. S. Pinto, A. Kolesnikov, X. Wang, D. Salz, M. Neumann, I. Alabdulmohsin, M. Tschannen, E. Bugliarello, et al. Paligemma: A versatile 3b vlm for transfer. *arXiv preprint arXiv:2407.07726*, 2024.
- [34] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*, volume 2024, pages 21875–21895, 2024.
- [35] T. Darcet, M. Oquab, J. Mairal, and P. Bojanowski. Vision transformers need registers. In *International conference on learning representations*, volume 2024, pages 2632–2652, 2024.

A Hardware platform

Demonstrations are collected on TeleAvatar, a commercial bimanual mobile manipulator from LingYu Robotics: two 7-DoF arms, each terminated by a one-DoF gripper, are mounted on a torso column above a wheeled mobile base.

Sensing. The robot carries three RGB cameras—a head-mounted stereo fisheye camera and one camera on each wrist—all recorded at 30 fps. The head camera is captured side-by-side at 2160×4320 (2160×2160 per eye) and the two wrist cameras at 480×848 . The head fisheye stream is rectified to an equirectangular panorama and presented to the operator as a stereo VR180 view (Appendix B); the policy consumes the left-cropped square head view (2160×2160) together with the two wrist views. The robot also reports proprioceptive joint state, which the policy consumes as its state input.

Teleoperation. The operator wears a Pico 4 Enterprise headset and holds its two controllers, whose poses are retargeted in real time to the two arms and grippers. The headset’s integrated eye tracker simultaneously records the operator’s gaze; Appendix B describes how this gaze stream is registered to the head-camera image.

B Gaze Data Collection

Capture. The operator’s gaze is recorded by the eye tracker integrated in the Pico 4 Enterprise headset, synchronized with the head-camera stream at 30 fps. At each timestep, the tracker emits a 3D gaze ray in the headset frame. Because the policy consumes head-camera pixels rather than headset-frame rays, we register every ray to head-camera image coordinates and Kalman-filter the resulting 2D gaze trace to reduce measurement noise.

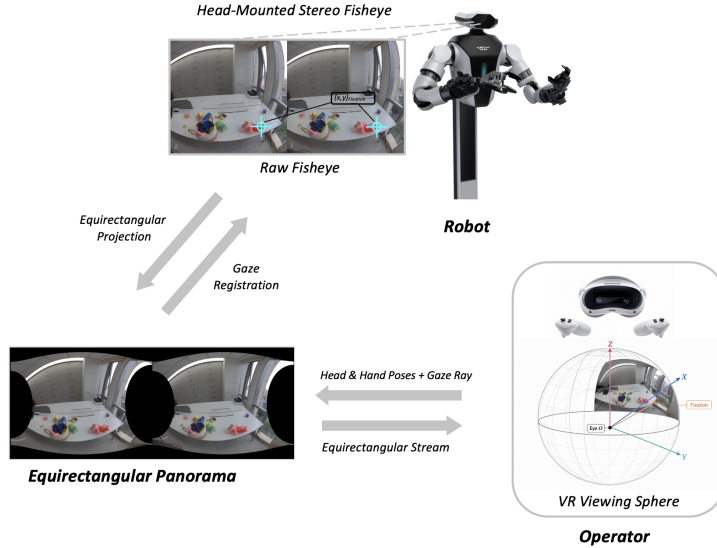


Figure 9: Gaze-to-image registration. The display path maps the head-camera fisheye image to an equirectangular panorama shown on the operator’s VR viewing sphere; the gaze-registration path inverts this chain to map the operator’s eye-tracker gaze to a 2D location on the head-camera image (`observation.gaze`).

Registration. The display path maps a raw fisheye head-camera image to an equirectangular panorama and then onto the operator’s VR viewing sphere. Registration inverts this chain. For each frame, we take the gaze direction in the headset frame, intersect this ray with the viewing

sphere, convert the intersection to equirectangular coordinates, and map that point back through the calibrated panorama-to-fisheye transformation to a pixel on the original head-camera image. We finally remap this pixel into the left-cropped square view consumed by the predictor and policy, and store the resulting 2D gaze location as `observation.gaze` (Figure 9).

Registration runs once, offline, during dataset conversion. The resulting 2D gaze locations supervise the gaze predictor and are rendered as visual prompts during policy fine-tuning. At deployment, the eye tracker is removed: the predictor estimates the gaze location from recent head-camera frames and the language instruction, and the predicted location is rendered in the same way. After dataset conversion, neither predictor training nor policy training/deployment requires headset pose, eye-tracker hardware, camera intrinsics, or viewing-sphere geometry.

C Task descriptions

We expand on the six GAZEMANI tasks summarized in §4.1. All tasks are collected on the platform described in Appendix A with 200 demonstrations each. Each task’s instruction is shown in italics.

T1: Stack Bowls. (*“stack red bowls” / “stack yellow bowls”*) Six same-sized bowls—three red and three yellow—are placed on the table: one red and one yellow at fixed positions near the robot, and the other two red and two yellow in a row farther away. The robot stacks the two far bowls of the instructed color onto the near bowl of that color. A trial is successful when both far bowls of the instructed color are stacked on the matching near bowl. As the bowls differ only in color, the task isolates instruction-conditioned instance selection.

T2: Stack Cubes in Order. (*“stack cubes in order: yellow first, then purple”*) A yellow cube and a purple cube are placed on the table together with a marked target ring. Using both arms, the robot stacks the cubes inside the ring in a fixed color order, the yellow cube first and the purple cube on top. A trial is successful when both cubes are stacked in the correct order within the ring. The per-phase change of target, together with the bimanual handoff, makes this a test of phase-dependent sequencing.

T3: Bus Table. (*“bus the table: place dishes, bowls, and cups into the bussing bin, and place trash into the trash bin”*) The workspace contains tableware—dishes, bowls, and cups—and pieces of trash, alongside a bussing bin and a trash bin. The robot sorts every item into its correct bin: tableware into the bussing bin, trash into the trash bin. A trial is successful when every item reaches its correct bin. With two destinations and many objects, the task stresses long-horizon target maintenance across an extended sequence.

T4: Fold Towel. (*“fold the towel”*) Starting from a flat towel, the robot folds it once, rotates it, and folds it twice more to finish. A trial is successful when the towel is fully folded through this sequence. As the relevant target is a deforming edge rather than a rigid object, the task tests deformable-state tracking.

T5: Insert Toilet Paper. (*“insert the toilet paper roll onto the spindle”*) A paper roll is placed at a randomized position on the table beside an upright spindle. The robot picks up the roll and threads it onto the spindle by aligning the roll’s central hole over the tip. A trial is successful when the roll is seated on the spindle. The small insertion target and the variable starting position make this a test of precise contact localization.

T6: Catch Rolling Ball. (*“catch the rolling ping-pong ball and place it into the bin”*) A ping-pong ball is released on an incline and rolls freely across the workspace. The robot must intercept the ball before it rolls out of reach and drop it into a bin. A trial is successful when the ball is caught in time and placed in the bin. Because the target moves and the grasp is time-critical, the task tests motion anticipation.

D Dataset statistics

Table 4 summarizes GAZEMANI: exactly 200 demonstrations per task, 1,200 trajectories in total, and 693,539 synchronized frames recorded at 30 fps (≈ 6.4 hours of teleoperation). Demonstrations are stored in the LeRobot format; every frame carries the full observation–action–gaze tuple described in Appendix A, including the registered 2D gaze location (`observation.gaze`). Mean episode length spans more than a factor of five across tasks—from the shorter episodes in T5 and T6 (~ 7 s) to the long-horizon sequences in T3 and T4 (35–40 s)—reflecting the differing task structure. Only T1 carries more than one language instruction (“stack red bowls” / “stack yellow bowls”), consistent with its role as the instruction-conditioned instance-selection task.

Table 4: GAZEMANI dataset statistics, per task and overall. Durations assume 30 fps; “Range” is the shortest–longest episode. “Instr.” is the number of distinct language instructions.

Task	Description	Instr.	Episodes	Frames	Mean (s)	Range (s)
T1	Stack Bowls	2	200	77,803	13.0	9.1–19.3
T2	Stack Cubes in Order	1	200	77,307	12.9	9.8–19.7
T3	Bus Table	1	200	211,295	35.2	24.2–70.4
T4	Fold Towel	1	200	237,817	39.6	27.3–65.7
T5	Insert Toilet Paper	1	200	45,288	7.5	5.7–11.5
T6	Catch Rolling Ball	1	200	44,029	7.3	6.0–10.2
Total			1,200	693,539	≈ 385 min (6.4 h)	

E Training and implementation details

Table 5 lists the hyperparameters for the gaze predictor and the per-task π_0 policies. Predictor and policy are trained independently, with no gradients flowing between them (§3.3; predictor architecture in §3.1).

Table 5: Predictor and per-task policy implementation details.

Gaze predictor (~ 2.9 M trainable parameters)	
Vision/text encoder	CLIP ViT-B/16 (<code>openai/clip-vit-base-patch16</code>), frozen
Fusion dimension	256
Temporal window	$N=3$ head frames at 224×224 (from the 2160-px left crop)
Prompt conditioning	per-frame vision→text cross-attention (4 heads) + FFN
Temporal fusion	2-layer transformer encoder (4 heads), $N \times 196$ tokens
Supervision	per-frame cross-entropy on Gaussian gaze heatmap (default σ : Appendix F)
Optimizer	AdamW, $\beta=(0.9, 0.999)$, weight decay 0.01
Learning rate	1×10^{-4} , cosine, 200 warm-up $\rightarrow 1 \times 10^{-6}$
Batch / precision	64 / bfloat16
Sampling	25–80 sampled timesteps per episode (scaled with episode length); task-balanced weighted sampler; 80/20 episode-level train/validation split
Training length	≤ 100 epochs, early stopping (patience 10)
Inference smoothing	causal EMA, $\alpha=0.2$
π_0 policy (full fine-tune, per task)	
Initialization	released π_0 base [3]: PaliGemma VLM + 300M action expert
Observations	head + two wrist views at 224×224 ; crosshair on head only (if any)
Optimizer	AdamW, $\beta=(0.9, 0.95)$, weight decay 1×10^{-10} , grad-norm clip 1.0
Learning rate	peak 2.5×10^{-5} , 1,000 warm-up, cosine decay over 30,000 steps (to 2.5×10^{-6} ; training stops at 20,000)
Batch / precision	64 / bfloat16
Weight EMA	decay 0.99
Training length	20,000 steps
Action head	chunk length 30
Denosing	10-step flow matching (Euler)

Unified multi-task policies. For the unified evaluation in Figure 6, each method is trained as a single policy on the pooled demonstrations from all six tasks rather than as six separate policies. We use the same policy hyperparameters as in the per-task experiments, including the optimizer, learning-rate schedule, batch size, and 20,000 training steps. We do not apply task-balanced reweighting when sampling the pooled multi-task data. Evaluation uses 20 rollouts per task, for 120 rollouts per method.

Aux-loss baseline. Aux-loss uses the Gaze-Regularized VLA objective [13], replacing its synthetic gaze with our recorded human gaze while keeping the original KL coefficient. Gaze supervision is used only during training; no gaze cue is provided to the aux-loss policy at inference.

Reproducibility. All reported policy cells use one training seed. Per-task success is evaluated over randomized real-robot rollouts as specified in the main text. The GAZEMANI dataset and evaluation code are available at <https://gazemani.github.io/>.

F Gaze predictor accuracy and smoothing ablation

The predictor is supervised with a Gaussian-target heatmap: ground-truth gaze is rasterized onto the predictor’s 224×224 output grid and smoothed with a Gaussian of standard deviation σ (measured in grid cells; one cell ≈ 9.6 px in the 2160-px head crop). Small σ produces a sharp target, whereas large σ produces a more spatially diffuse target.

We sweep $\sigma \in \{4, 8, 16, 24\}$ on the joint six-task predictor, each an independent fine-tune from the same initialization on the same demonstrations with all other hyperparameters fixed. Table 6 reports predictor pixel-L2 error in the 2160-px head-crop coordinates, evaluated with causal EMA smoothing ($\alpha=0.2$) and the soft-argmax readout on the held-out validation episodes of each task.

Table 6: Effect of gaze-heatmap smoothing σ on predictor pixel-L2 error (px, lower is better) per task. $\sigma=16$ minimizes the mean and is used in all main-paper experiments.

σ	Catch	Insert	Bowls	Cubes	Bus	Towel	Mean
4	180.3	94.4	79.2	105.4	98.0	62.5	103.3
8	179.3	95.7	70.7	117.5	94.7	54.1	102.0
16	165.8	90.6	65.3	105.3	92.4	61.5	96.8
24	167.2	99.2	67.1	111.2	90.0	62.2	99.5

With the default $\sigma=16$, the mean localization error is 96.8 px in the 2160-px head crop, corresponding to approximately 10.0 px after resizing to the policy’s 224×224 input. This is smaller than the rendered crosshair half-arm of approximately 12 px at policy resolution (Appendix G). The dynamic Catch task is the hardest for the predictor, while the remaining tasks have lower localization error.

The mean error is U-shaped across the tested smoothing values: $\sigma=16$ minimizes the average and is best on four of the six tasks (Catch, Insert, Bowls, Cubes); Bus is best at $\sigma=24$ by 2.4 px and Towel at $\sigma=8$ by 7.4 px. Validation cross-entropy is not compared across σ , because changing the target width changes the entropy of the target distribution itself; we therefore report pixel-L2 localization error.

G Visual-prompt rendering details

The gaze prompt is rendered as a fixed crosshair centered at the 2D gaze location on the head-camera image. The prompt is applied only to the head view, since gaze is registered in the head-camera coordinate system; the two wrist-camera views are passed to the policy unchanged.

The same crosshair geometry and appearance are used across all prompted variants, tasks, and instructions: a cross with a small center circle, drawn with a half-arm of 120 px in the 2160-px head crop (≈ 12 px in the policy’s 224×224 input), with fixed opacity and a fixed cyan color, RGB (0, 255, 255).

The color is not conditioned on task identity, instruction, object class, cue source, or predictor confidence. Thus, the prompt supplies spatial information through its location rather than semantic information through task-specific appearance. We use a multi-pixel cross rather than a single-pixel dot so that the cue remains visible after image resizing and visual-token embedding.

H Cross-attention extraction from π_0

π_0 uses the PaliGemma [33] backbone, in which image patch tokens and language tokens are processed by a single self-attention stack; the “cross-attention” we visualize is the sub-block of that self-attention whose queries are the instruction tokens and whose keys are the head-camera (base_0_rgb) visual tokens. The SigLIP encoder produces a 16×16 grid of visual tokens per view, and we use only the head-camera block since the wrist views carry no crosshair. We use the instruction (language) tokens as queries, which directly probe how the instruction grounds to image regions.

We aggregate attention over the intermediate decoder layers $\ell \in \{8, \dots, 13\}$ of the 18-layer Gemma backbone, a band fixed on the vanilla policy and applied identically to all methods. Early layers are close to uniform, and late layers ($\ell \geq 14$) collapse onto attention-sink / register tokens [34, 35] and produce sharp but content-independent maps; the intermediate band is where instruction-conditioned spatial grounding concentrates.

At each frame we run one forward pass on the head-view RGB observation and the tokenized instruction, average the post-softmax weights over query heads and over the real instruction tokens, restrict to the head-camera key columns, average over the layer band, and reshape to 16×16 .

I Position-based cue baselines

The cue-source ablation in §4.4 compares predicted human gaze with position-based cues on T6 (*Catch Rolling Ball*) while keeping the input-pixel rendering route fixed. All prompted variants use the same crosshair geometry and appearance (Appendix G).

Current-position cue. We use a CPU color-blob detector on the policy’s 224×224 head-camera input and render the crosshair at the detected ball center in the current frame. The detector runs in 4.06 ms per frame, close to the gaze predictor’s 3.8 ms latency. In a manual check of 200 pre-grasp frames, the rendered cue lies on the ball in 94.5% of frames (Wilson 95% CI [90.4, 96.9]).

Kalman future-position cue. We track the detected ball center with a constant-velocity Kalman filter with state $\mathbf{x} = [x, y, v_x, v_y]^\top$, where position is represented in pixels in the 2160×2160 left-cropped head view and velocity in pixels per second. Measurement noise is fixed to $\mathbf{R} = \text{diag}(16, 9)$, estimated from detector jitter using a robust MAD estimate (approximately 4 and 3 px standard deviation along the two image axes). Process noise follows the standard white-acceleration constant-velocity model with $\sigma_a = 800$; this value is selected on demonstration trajectories to minimize 541 ms extrapolation error.

Measurements whose innovation exceeds 200 pixels are rejected to suppress occasional detector jumps to distractors, missing measurements are coasted for at most 10 frames, and the actual inter-frame interval is supplied at every update rather than assuming a fixed frame rate. The future cue is obtained by propagating the filtered state 0.541 s forward and rendering the crosshair at the predicted position. The same implementation is used to precompute training cues and to generate cues online at inference. An equivalence check over 30 episodes gives a maximum discrepancy of 0.706 px between the two pipelines, attributable to serialization rounding.

Random and no-cue controls. The random-crosshair control samples the cue location uniformly over the full 224×224 head-camera input, preserving the presence and appearance of the visual mark while removing task-relevant spatial information. The no-cue control uses the unmodified head-camera image.

J Predictor latency

The gaze predictor adds one streaming forward pass per policy step. We benchmark wall-clock latency on a single NVIDIA RTX 4090, averaging over 100 warmed-up iterations. Its streaming forward pass costs 3.8 ± 0.1 ms per frame at steady state, using a rolling cache for the recent frame window and encoding the instruction once.

A single π_0 policy step takes 70.5 ± 3.0 ms on the same deployment setup. Run serially, gaze prediction, crosshair rendering, and the policy require approximately 75 ms per step. In deployment, gaze prediction and rendering run on a background thread off the policy’s critical path, so policy execution remains the dominant latency.

For the T6 cue-source comparison, both position-based cues use the CPU color-blob detector described in Appendix I. The detector takes 4.06 ms per frame on the policy’s 224×224 input, closely matching the gaze predictor’s 3.8 ms cost. This keeps cue-generation latency comparable when contrasting predicted gaze with current- and future-position cues.

K Anticipation timing analysis

We measure gaze lead relative to recorded robot actions in the T6 and T3 demonstrations. Robot events are identified automatically from proprioception; gaze lead is computed from the registered head-camera gaze trace.

T6: Catch Rolling Ball. Arm-movement onset, t_{move} , is the first of three consecutive frames in which the maximum absolute right-arm joint velocity exceeds 0.5; it is detected in all 200 demonstrations. Let $g_0 = g(t_{\text{move}})$ denote the gaze location at movement onset. We trace backward to the beginning of the final continuous dwell within $R = 200$ px of g_0 on the 2160×2160 head-camera view. The mean observed lead is 949 ms ($n = 181$); in the remaining 19 episodes, the dwell is already ongoing at the beginning of the recording and its onset cannot be determined.

The ball reaches the movement-onset gaze location on average 541 ms after arm movement begins, setting the 0.541 s horizon of the Kalman future-position cue. In a separate timing analysis, we predict the same ball-arrival event from gaze-dwell onset and from movement onset, using the corresponding event-specific horizons. On the same episodes, the mean prediction errors are 420 and 118 px, respectively.

T3: Bus Table. Gripper closure and opening are detected automatically from the gripper state. The next-object and placement-location anchors are determined from image differences around each event, independently of gaze. For each anchor, we measure the interval from the onset of the final sustained gaze dwell within $R = 150$ px to the corresponding gripper event. A dwell requires at least five consecutive frames.

The median lead is 1667 ms before gripper closure for the next object ($n = 1194$) and 933 ms before gripper opening for the placement location ($n = 1093$). The former interval can include visually guided reaching; at the latter time, the manipulated object has not yet reached the placement location.

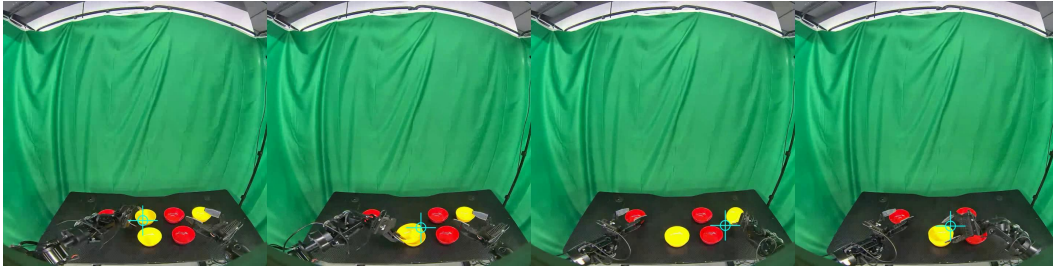
L Qualitative task rollouts

The following figures show representative rollouts of gaze-prompt π_0 (ours) on all six tasks. The cyan crosshair on the head-camera view is the rendered gaze prompt that the policy receives as input. Frames run left to right (top row first for the two-row strips). Uncut single-take recordings of all rollouts for every task and method are available on the project website.

T1: Stack Bowls (“*stack red bowls*”)



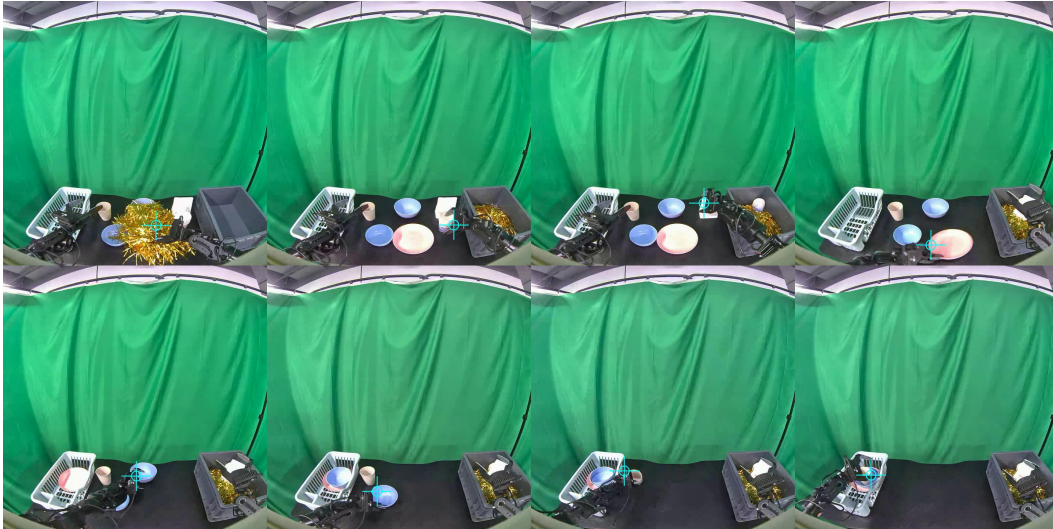
T1: Stack Bowls (“*stack yellow bowls*”)



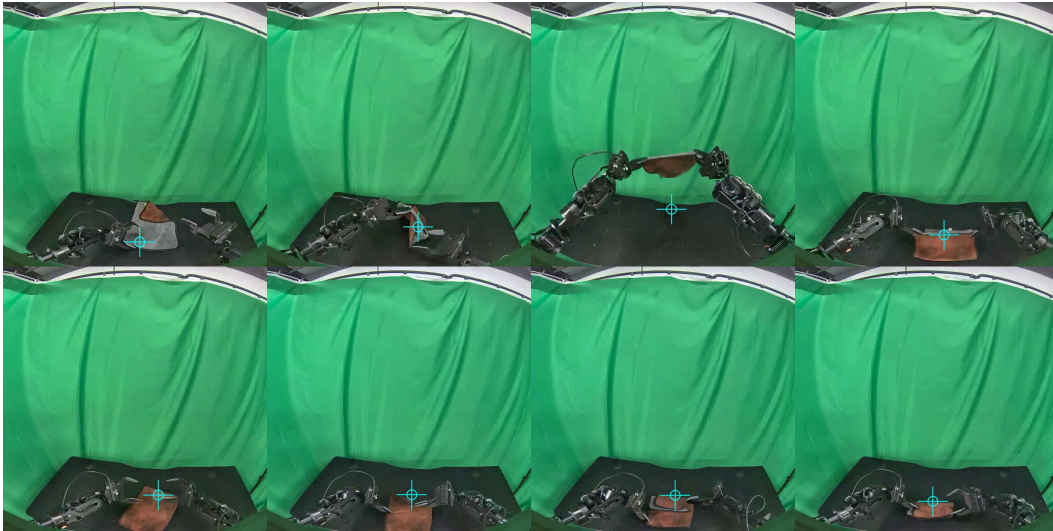
T2: Stack Cubes in Order



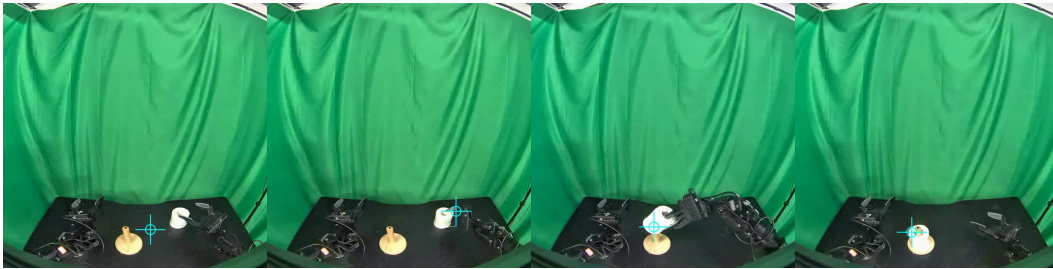
T3: Bus Table



T4: Fold Towel



T5: Insert Toilet Paper



T6: Catch Rolling Ball



M Visual robustness

We evaluate T1 under two visual changes relative to the data-collection setup. First, we add previously unseen distractor objects at randomized positions. Gaze-prompt reaches 68% success, compared with 40% for vanilla π_0 ($n=50$ rollouts per method, $p=0.009$). Second, we remove the green backdrop used during data collection. Gaze-prompt reaches 66% success, compared with 22% for vanilla ($n=50$, $p=2 \times 10^{-5}$). For reference, gaze-prompt reaches 72% and vanilla 38% in the unperturbed T1 evaluation.

These tests show that the performance gain persists under the two evaluated visual changes. They do not establish generalization to arbitrary lighting, unseen object instances, or new environments.